

```
from google.colab import files
uploaded = files.upload()
```

No file chosen  
this cell to enable

Upload widget is only available when the cell has been executed in the current browser session. Please rerun

```
import pandas as pd

df = pd.read_csv("1 iris.csv")
df.head()
```

|   | sepal_length | sepal_width | petal_length | petal_width | species |
|---|--------------|-------------|--------------|-------------|---------|
| 0 | 5.1          | 3.5         | 1.4          | 0.2         | setosa  |
| 1 | 4.9          | 3.0         | 1.4          | 0.2         | setosa  |
| 2 | 4.7          | 3.2         | 1.3          | 0.2         | setosa  |
| 3 | 4.6          | 3.1         | 1.5          | 0.2         | setosa  |
| 4 | 5.0          | 3.6         | 1.4          | 0.2         | setosa  |

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   sepal_length    150 non-null   float64
1   sepal_width     150 non-null   float64
2   petal_length    150 non-null   float64
3   petal_width     150 non-null   float64
4   species         150 non-null   object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

```
df.isnull().sum()
```

```
0
sepal_length  0
sepal_width   0
petal_length  0
petal_width   0
species       0

dtype: int64
```

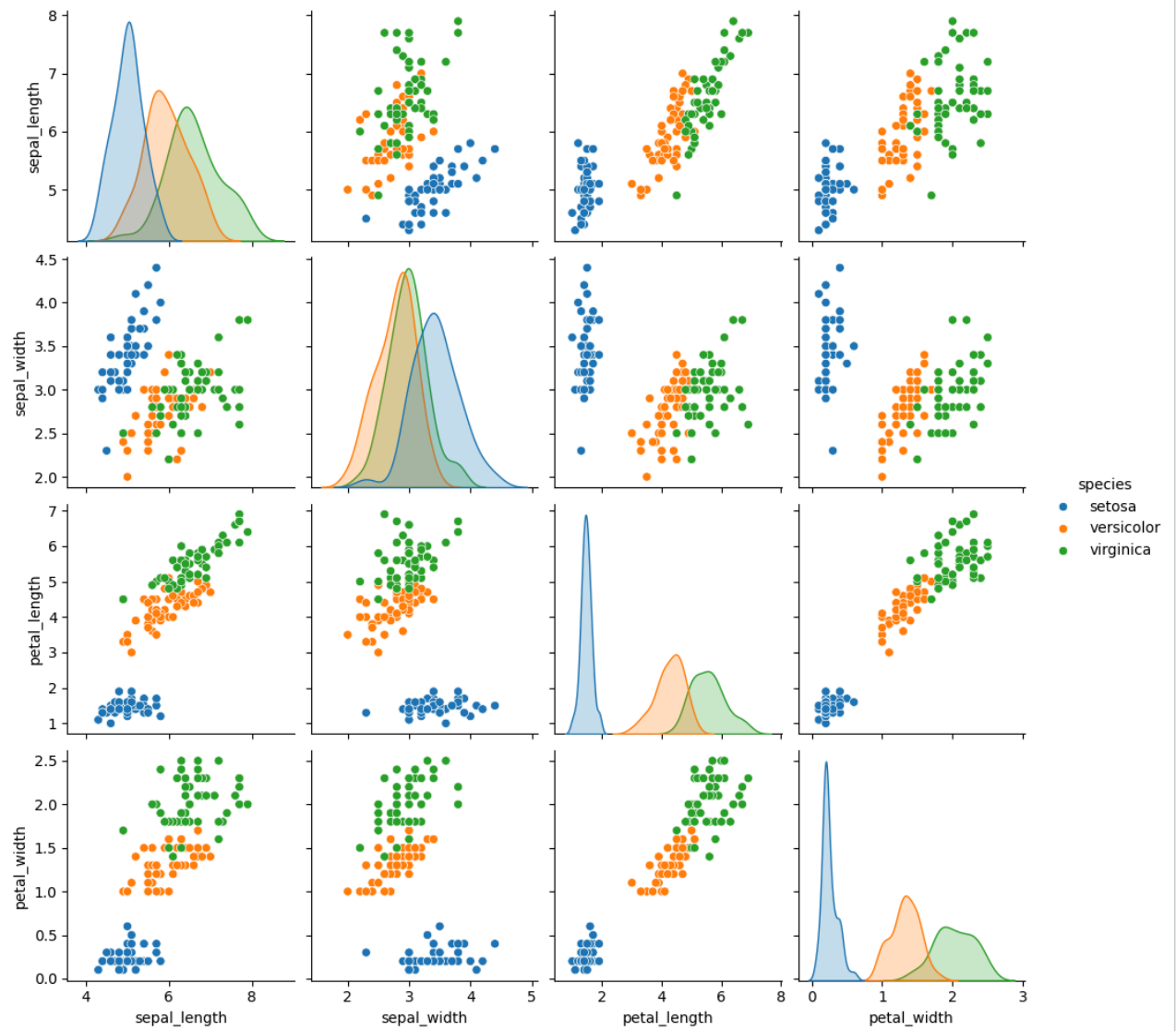
```
df.describe()
```

|       | sepal_length | sepal_width | petal_length | petal_width |
|-------|--------------|-------------|--------------|-------------|
| count | 150.000000   | 150.000000  | 150.000000   | 150.000000  |
| mean  | 5.843333     | 3.054000    | 3.758667     | 1.198667    |
| std   | 0.828066     | 0.433594    | 1.764420     | 0.763161    |
| min   | 4.300000     | 2.000000    | 1.000000     | 0.100000    |
| 25%   | 5.100000     | 2.800000    | 1.600000     | 0.300000    |
| 50%   | 5.800000     | 3.000000    | 4.350000     | 1.300000    |
| 75%   | 6.400000     | 3.300000    | 5.100000     | 1.800000    |
| max   | 7.900000     | 4.400000    | 6.900000     | 2.500000    |

```
df.drop_duplicates(inplace=True)
```

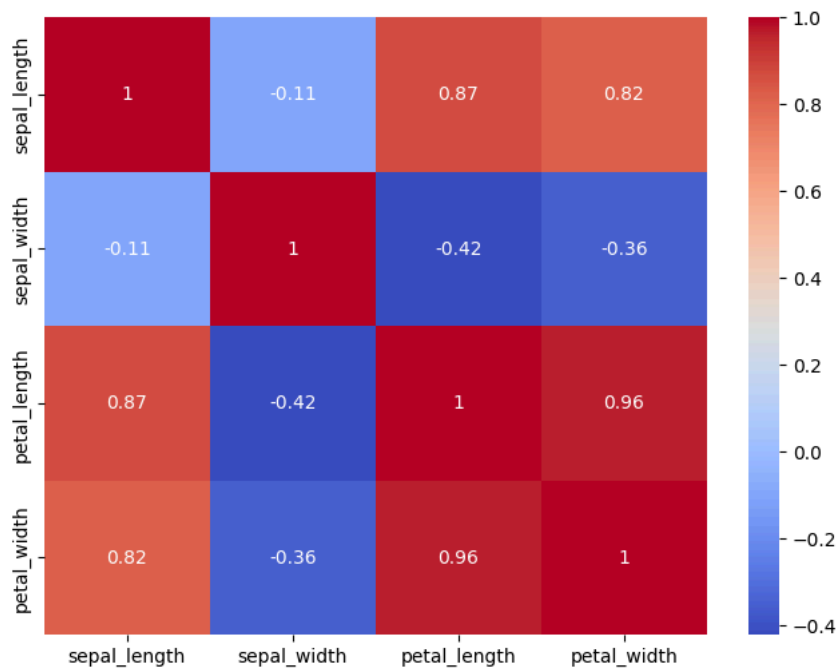
```
import seaborn as sns
import matplotlib.pyplot as plt

sns.pairplot(df, hue='species')
plt.show()
```

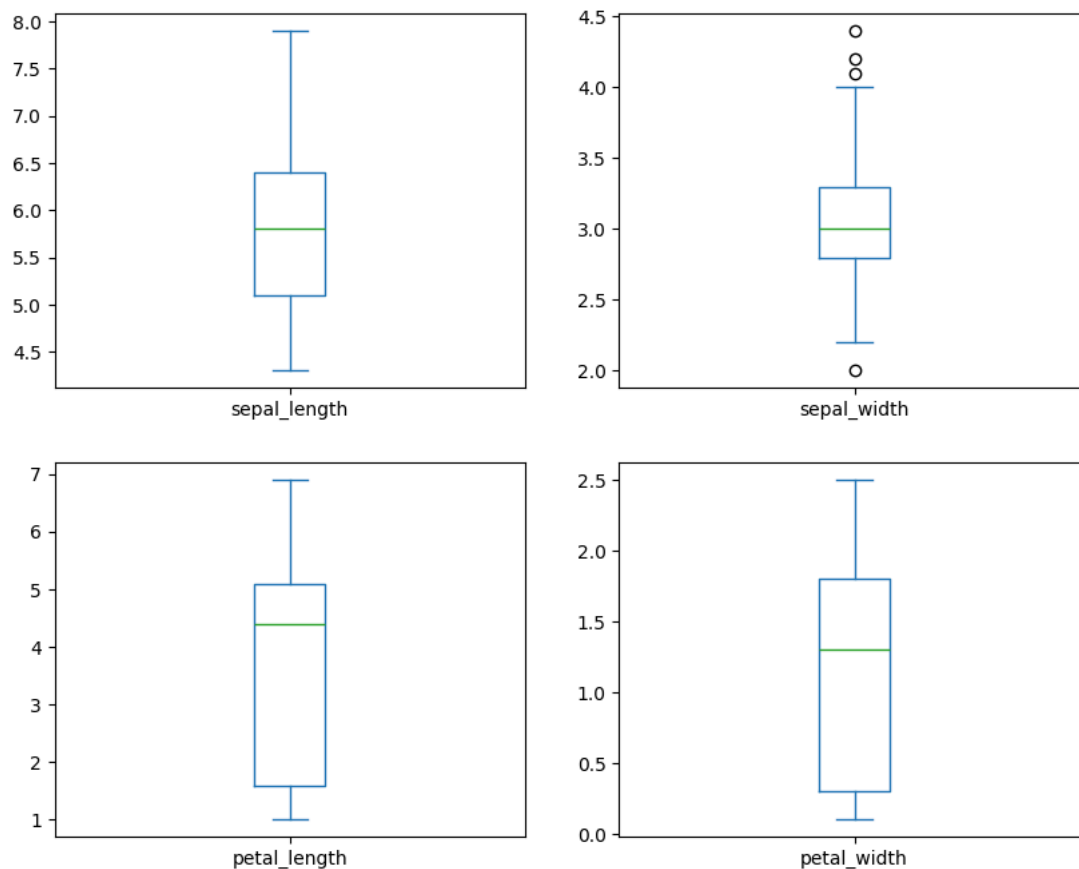


```
numeric_df = df.select_dtypes(include=['float64', 'int64'])
```

```
plt.figure(figsize=(8,6))
sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm')
plt.show()
```



```
df.plot(kind='box', subplots=True, layout=(2,2), figsize=(10,8))
plt.show()
```



```
df.to_csv("cleaned_iris.csv", index=False)
```

```
from google.colab import files
files.download("cleaned_iris.csv")
```

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

```
X = df.drop("species", axis=1)
y = df["species"]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = RandomForestClassifier()
model.fit(X_train, y_train)

predictions = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, predictions))
```

Accuracy: 0.9333333333333333

```
from google.colab import files
files.upload()
```

[Show hidden output](#)

```
import pandas as pd

df = pd.read_csv("4) house Prediction Data Set.csv")
df.head()
```

```
0.00632 18.00 2.310 0 0.5380 6.5750 65.20 4.0900 1 296.0 15.30 396.90 4.98 24.00

0      0.02731 0.00 7.070 0 0.4690 6.4210 78...
1      0.02729 0.00 7.070 0 0.4690 7.1850 61...
2      0.03237 0.00 2.180 0 0.4580 6.9980 45...
3      0.06905 0.00 2.180 0 0.4580 7.1470 54...
4      0.02985 0.00 2.180 0 0.4580 6.4300 58...
```

```
df.info()
df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 505 entries, 0 to 504
Data columns (total 1 columns):
#   Column
---  ---
0   0.00632 18.00 2.310 0 0.5380 6.5750 65.20 4.0900 1 296.0 15.30 396.90 4.98 24.00
dtypes: object(1)
memory usage: 4.1+ KB

0.00632 18.00 2.310 0 0.5380 6.5750 65.20 4.0900 1 296.0 15.30 396.90 4.98 24.00

0      0.02731 0.00 7.070 0 0.4690 6.4210 78...
1      0.02729 0.00 7.070 0 0.4690 7.1850 61...
2      0.03237 0.00 2.180 0 0.4580 6.9980 45...
3      0.06905 0.00 2.180 0 0.4580 7.1470 54...
4      0.02985 0.00 2.180 0 0.4580 6.4300 58...
```

```
df = pd.read_csv("4) house Prediction Data Set.csv", delim_whitespace=True)
df.head()
```

```
/tmp/ipykernel_3212/2029134270.py:1: FutureWarning: The 'delim_whitespace' keyword in pd.read_csv is deprecated and will be
df = pd.read_csv("4) house Prediction Data Set.csv", delim_whitespace=True)

0.00632 18.00 2.310 0 0.5380 6.5750 65.20 4.0900 1 296.0 15.30 396.90 4.98 24.00

0 0.02731 0.0 7.07 0 0.469 6.421 78.9 4.9671 2 242.0 17.8 396.90 9.14 21.6
1 0.02729 0.0 7.07 0 0.469 7.185 61.1 4.9671 2 242.0 17.8 392.83 4.03 34.7
2 0.03237 0.0 2.18 0 0.458 6.998 45.8 6.0622 3 222.0 18.7 394.63 2.94 33.4
3 0.06905 0.0 2.18 0 0.458 7.147 54.2 6.0622 3 222.0 18.7 396.90 5.33 36.2
4 0.02985 0.0 2.18 0 0.458 6.430 58.7 6.0622 3 222.0 18.7 394.12 5.21 28.7
```

```
df.columns = [
    "CRIM", "ZN", "INDUS", "CHAS", "NOX", "RM", "AGE",
    "DIS", "RAD", "TAX", "PTRATIO", "B", "LSTAT", "PRICE"
]
```

```
df.info()
df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 505 entries, 0 to 504
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    CRIM        505 non-null    float64
1    ZN          505 non-null    float64
2    INDUS       505 non-null    float64
3    CHAS        505 non-null    int64
4    NOX         505 non-null    float64
5    RM          505 non-null    float64
6    AGE         505 non-null    float64
7    DIS         505 non-null    float64
8    RAD         505 non-null    int64
9    TAX         505 non-null    float64
10   PTRATIO     505 non-null    float64
11   B           505 non-null    float64
12   LSTAT       505 non-null    float64
13   PRICE       505 non-null    float64
dtypes: float64(12), int64(2)
memory usage: 55.4 KB
```

|   | CRIM    | ZN  | INDUS | CHAS | NOX   | RM    | AGE  | DIS    | RAD | TAX   | PTRATIO | B      | LSTAT | PRICE |
|---|---------|-----|-------|------|-------|-------|------|--------|-----|-------|---------|--------|-------|-------|
| 0 | 0.02731 | 0.0 | 7.07  | 0    | 0.469 | 6.421 | 78.9 | 4.9671 | 2   | 242.0 | 17.8    | 396.90 | 9.14  | 21.6  |
| 1 | 0.02729 | 0.0 | 7.07  | 0    | 0.469 | 7.185 | 61.1 | 4.9671 | 2   | 242.0 | 17.8    | 392.83 | 4.03  | 34.7  |
| 2 | 0.03237 | 0.0 | 2.18  | 0    | 0.458 | 6.998 | 45.8 | 6.0622 | 3   | 222.0 | 18.7    | 394.63 | 2.94  | 33.4  |
| 3 | 0.06905 | 0.0 | 2.18  | 0    | 0.458 | 7.147 | 54.2 | 6.0622 | 3   | 222.0 | 18.7    | 396.90 | 5.33  | 36.2  |
| 4 | 0.02985 | 0.0 | 2.18  | 0    | 0.458 | 6.430 | 58.7 | 6.0622 | 3   | 222.0 | 18.7    | 394.12 | 5.21  | 28.7  |

```
from sklearn.model_selection import train_test_split
```

```
X = df.drop("PRICE", axis=1)
y = df["PRICE"]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
from sklearn.model_selection import train_test_split
```

```
X = df.drop("PRICE", axis=1)
y = df["PRICE"]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
model.fit(X_train, y_train)
```

```
LinearRegression
```

```
pred = model.predict(X_test)
```

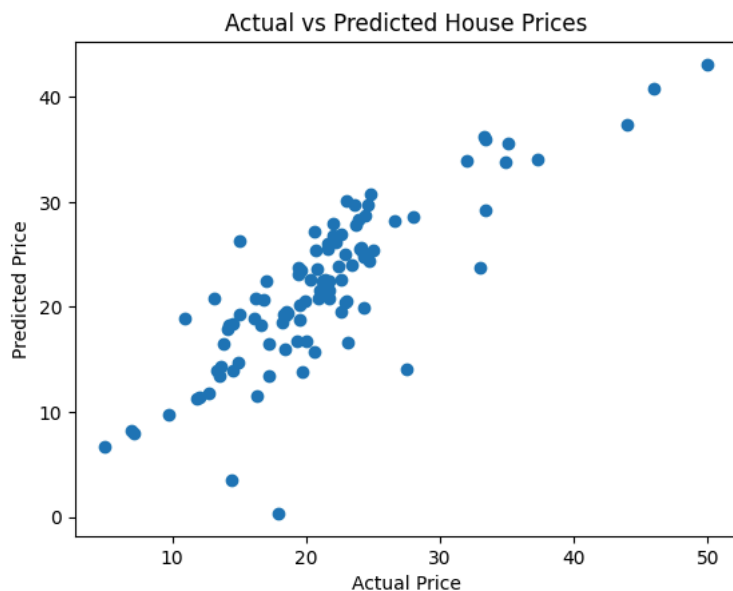
```
from sklearn.metrics import mean_squared_error, r2_score
```

```
print("MSE:", mean_squared_error(y_test, pred))
print("R2 Score:", r2_score(y_test, pred))
```

```
MSE: 19.379041116137028
R2 Score: 0.652220188866615
```

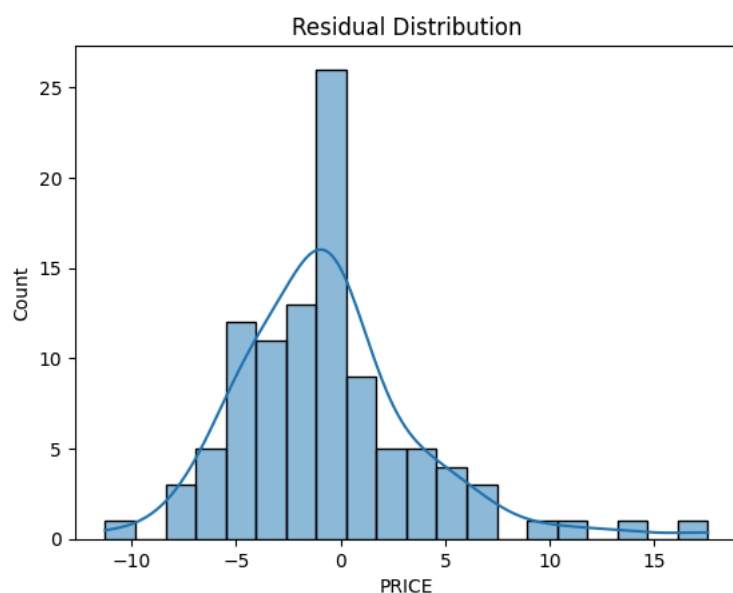
```
import matplotlib.pyplot as plt
```

```
plt.scatter(y_test, pred)
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted House Prices")
plt.show()
```



```
import seaborn as sns

sns.histplot(y_test - pred, bins=20, kde=True)
plt.title("Residual Distribution")
plt.show()
```



```
df_result = pd.DataFrame({
    "Actual": y_test,
    "Predicted": pred
})

df_result.to_csv("house_predictions.csv", index=False)
```

```
from google.colab import files
files.download("house_predictions.csv")
```

```
import pandas as pd

df = pd.read_csv("Tweets.csv")

df.head()
```

```
from google.colab import files
files.upload()
```

[Show hidden output](#)

```
import pandas as pd

df = pd.read_csv("Twitter_Data.csv")

df.head()
```

|   | clean_text                                        | category |
|---|---------------------------------------------------|----------|
| 0 | when modi promised "minimum government maximum... | -1.0     |
| 1 | talk all the nonsense and continue all the dra... | 0.0      |
| 2 | what did just say vote for modi welcome bjp t...  | 1.0      |
| 3 | asking his supporters prefix chowkidar their n... | 1.0      |
| 4 | answer who among these the most powerful world... | 1.0      |

```
import pandas as pd
import re
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, confusion_matrix

# Load dataset
df = pd.read_csv("Twitter_Data.csv")

# Show first rows
print(df.head())

# Clean text
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'http\S+', '', text)
    text = re.sub(r'^a-zA-Z\s', '', text)
    return text

df['clean_text'] = df['clean_text'].apply(clean_text)

# Drop rows where 'category' is NaN
df.dropna(subset=['category'], inplace=True)

# Convert text to numbers
vectorizer = CountVectorizer()

X = vectorizer.fit_transform(df['clean_text'])

y = df['category']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
model = MultinomialNB()

model.fit(X_train, y_train)

# Predictions
predictions = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, predictions)

print("Accuracy:", accuracy)

# Confusion matrix
cm = confusion_matrix(y_test, predictions)

sns.heatmap(cm, annot=True, fmt='d')

plt.title("Confusion Matrix")

plt.show()

# Save cleaned dataset
df.to_csv("cleaned_sentiment.csv", index=False)
```

```
print("Project Completed Successfully")
```

|   | clean_text                                        | category |
|---|---------------------------------------------------|----------|
| 0 | when modi promised "minimum government maximum... | -1.0     |
| 1 | talk all the nonsense and continue all the dra... | 0.0      |
| 2 | what did just say vote for modi welcome bjp t...  | 1.0      |
| 3 | asking his supporters prefix chowkidar their n... | 1.0      |
| 4 | answer who among these the most powerful world... | 1.0      |

Accuracy: 0.7515263077159073

